Week 3 – Greedy Algorithm:

1.Problem Statement

Given an array of 1s and 0s this has all 1s first followed by all 0s. Aim is to find the number of 0s. Write a program using Divide and Conquer to Count the number of zeroes in the given array.

Input Format

First Line Contains Integer m – Size of array

Next m lines Contains m numbers – Elements of an array

Output Format

First Line Contains Integer – Number of zeroes present in the given array.

Answer:

```c
#include <stdio.h>

int countZeroes(int a[], int low, int high) {
    if (low > high)
        return 0;

    int mid = (low + high) / 2;

    if (a[mid] == 0)
        return (high - mid + 1) + countZeroes(a, low, mid - 1);

    return countZeroes(a, mid + 1, high);
}

int main() {
    int m;
    scanf("%d", &m);

    int a[m];

    for (int i = 0; i < m; i++)
        scanf("%d", &a[i]);

    printf("%d", countZeroes(a, 0, m - 1));

    return 0;
}
```

	Input	Expected	Got	
✔	5 1 1 1 0 0	2	2	✔
✔	10 1 1 1 1 1 1 1 1 1 1	0	0	✔

Open block c

2.Given an array nums of size n, return *the majority element*.

The majority element is the element that appears more than ⌊n / 2⌋ times. You may assume that the majority element always exists in the array.

Example 1:

Input: nums = [3,2,3]
Output: 3

Example 2:

Input: nums = [2,2,1,1,1,2,2]
Output: 2

Constraints:

n == nums.length

1 <= n <= 5 * 104

-231 <= nums[i] <= 231 - 1

Answer:

```c
#include <stdio.h>

int majorityElement(int nums[], int low, int high) {
    if (low == high)
        return nums[low];

    int mid = (low + high) / 2;

    int left = majorityElement(nums, low, mid);
    int right = majorityElement(nums, mid + 1, high);

    if (left == right)
        return left;

    int leftCount = 0, rightCount = 0;

    for (int i = low; i <= high; i++) {
        if (nums[i] == left)
            leftCount++;
        if (nums[i] == right)
            rightCount++;
    }

    return leftCount > rightCount ? left : right;
}

int main() {
    int n;
    scanf("%d", &n);

    int nums[n];

    for (int i = 0; i < n; i++)
        scanf("%d", &nums[i]);

    printf("%d", majorityElement(nums, 0, n - 1));

    return 0;
}

```

	Input	Expected	Got	
✔	3 3 2 3	3	3	✔

Passed all tests! ✔

3. **Problem Statement:**

Given a sorted array and a value x, the floor of x is the largest element in array smaller than or equal to x. Write divide and conquer algorithm to find floor of x.

Input Format

First Line Contains Integer n – Size of array

Next n lines Contains n numbers – Elements of an array

Last Line Contains Integer x – Value for x

Output Format

First Line Contains Integer – Floor value for x

Answer:

```c
#include <stdio.h>

int floorValue(int a[], int low, int high, int x) {
    if (low > high)
        return -1;

    int mid = (low + high) / 2;

    if (a[mid] == x)
        return a[mid];

    if (a[mid] > x)
        return floorValue(a, low, mid - 1, x);

    int result = floorValue(a, mid + 1, high, x);

    if (result == -1)
        return a[mid];

    return result;
}

int main() {
    int n, x;
    scanf("%d", &n);

    int a[n];

    for (int i = 0; i < n; i++)
        scanf("%d", &a[i]);

    scanf("%d", &x);

    printf("%d", floorValue(a, 0, n - 1, x));

    return 0;
}
```

	Input	Expected	Got	
✔	6 1 2 8 10 12 19 5	2	2	✔
✔	5 10 22 85 108 129 100	85	85	✔
✔	7 3 5 7 9 11 13 15 10	9	9	✔

Passed all tests! ✔

4. **Problem Statement:**

Given a sorted array of integers say arr[] and a number x. Write a recursive program using divide and conquer strategy to check if there exist two elements in the array whose sum = x. If there exist such two elements then return the numbers, otherwise print as “No”.

Note: Write a Divide and Conquer Solution

Input Format

First Line Contains Integer n – Size of array

Next n lines Contains n numbers – Elements of an array

Last Line Contains Integer x – Sum Value

Output Format

First Line Contains Integer – Element1

Second Line Contains Integer – Element2 (Element 1 and Elements 2 together sums to value “x”)

Answer:

```c
#include <stdio.h>

int findPair(int a[], int left, int right, int x, int *p, int *q) {
    if (left >= right)
        return 0;

    int sum = a[left] + a[right];

    if (sum == x) {
        *p = a[left];
        *q = a[right];
        return 1;
    }

    if (sum < x)
        return findPair(a, left + 1, right, x, p, q);

    return findPair(a, left, right - 1, x, p, q);
}

int main() {
    int n, x, p, q;

    scanf("%d", &n);

    int a[n];

    for (int i = 0; i < n; i++)
        scanf("%d", &a[i]);

    scanf("%d", &x);

    if (findPair(a, 0, n - 1, x, &p, &q)) {
        printf("%d\n", p);
        printf("%d\n", q);
    } else {
        printf("No");
    }

    return 0;
}
```

	Input	Expected	Got	
✔	4 2 4 8 10 14	4 10	4 10	✔
✔	5 2 4 6 8 10 100	No	No	✔

Passed all tests! ✔

5. Write a Program to Implement the Quick Sort Algorithm

Input Format:
The first line contains the no of elements in the list-n
The next n lines contain the elements.

Output:
Sorted list of elements

Answer:

```c
#include <stdio.h>

void quickSort(int a[], int low, int high) {
    if (low < high) {
        int pivot = a[high];
        int i = low - 1;

        for (int j = low; j < high; j++) {
            if (a[j] <= pivot) {
                i++;
                int temp = a[i];
                a[i] = a[j];
                a[j] = temp;
            }
        }

        int temp = a[i + 1];
        a[i + 1] = a[high];
        a[high] = temp;

        int p = i + 1;

        quickSort(a, low, p - 1);
        quickSort(a, p + 1, high);
    }
}

int main() {
    int n;
    scanf("%d", &n);

    int a[n];

    for (int i = 0; i < n; i++)
        scanf("%d", &a[i]);

    quickSort(a, 0, n - 1);

    for (int i = 0; i < n; i++) {
        if (i > 0)
            printf(" ");
        printf("%d", a[i]);
    }

    return 0;
}
```

	Input	Expected	Got	
✔	5 67 34 12 98 78	12 34 67 78 98	12 34 67 78 98	✔
✔	10 1 56 78 90 32 56 11 10 90 114	1 10 11 32 56 56 78 90 90 114	1 10 11 32 56 56 78 90 90 114	✔
✔	12 9 8 7 6 5 4 3 2 1 10 11 90	1 2 3 4 5 6 7 8 9 10 11 90	1 2 3 4 5 6 7 8 9 10 11 90	✔